Provably Fair

Introduction

This document describes Provably Fair RNG implementation for single-player and multiplayer games.

Single-player

Provable fairness in single-player games relies on combining a *Player Seed* with a pre-committed *Casino Seed*. Thanks to casino's Cryptographic Commitment, players can independently verify that game outcomes are fair and have not been tampered with by the casino.

Generating Hash Buffer

For each single-player Round, subsequent hashes used for extracting random numbers are generated with the following formula:

`HMAC_SHA256(SERVER_SEED,$CLIENT_SEED:$NONCE:$HASH_INDEX)`

where *Hash Index* is incremented for as many extractions as the Round requires (see: General Concepts / Hash Buffer).

Server seed

The server seed is a generated 64-character hexadecimal string created by our system. Before placing any bets, players are provided with an encrypted hash of this server seed. This ensures the server seed cannot be altered by the casino operator, and that players cannot predict the results in advance.

To reveal the server seed from its hashed version, players must rotate the seed, prompting the system to generate a new one. At this point, you can verify that the hashed server seed matches the un-hashed version. This verification can be done by hashing the seed as a value (`HASH_SHA256($SERVER_SEED)`).

Client seed

The client seed is a string with up to 64 characters controlled by the player, ensures that players have an influence on the randomness of the outcomes. Without it, the server seed would solely determine the outcome of each bet. Players are encouraged to regularly change their client seed to create a new sequence of random outcomes. This process gives players control over the result generation, similar to cutting the deck in a brick-and-mortar casino.

During the first authentication, a client seed is automatically created to provide a seamless initial experience. While this randomly generated client seed is adequate, we strongly recommend choosing your own to incorporate your influence into the randomness.

Nonce

The nonce is a number that increments with each new round. Because of the nature of the SHA256 cryptographic function, this ensures a completely new result is generated every time, without the need to create new client and server seeds. The use of a nonce allows us to remain committed to your existing client and server seed pair while still generating unique results for each round.

Implementation

node.js

```
function random(serverSeed, clientSeed, nonce, cursor) {
    // Generate hmac
    const hashIndex = Math.floor(cursor / 8);
    const hmac = crypto.createHmac("sha256", serverSeed).update(`${clientSeed}:${nonce}:${hashIndex}`);

    // Read integer from the buffer
    const offset = cursor % 8;
    const result = hmac.digest().readUInt32BE(offset * 4);

    return result;
}
```

JavaScript in the Browser

Requires installing `crypto-js` (https://github.com/brix/crypto-js)

```
import CryptoJS from "crypto-js";

function random(serverSeed, clientSeed, nonce, cursor) {
    // Generate hmac
    const hashIndex = Math.floor(cursor / 8);
    const hmac = CryptoJS.HmacSHA256(`${clientSeed}:${nonce}:${hashIndex}`, serverSeed);

    // Read the integer and conver from signed to unsigned
    const offset = cursor % 8;
    const result = hmac.words[offset] >>> 0;

    return result;
}
```

Multiplayer

Provable fairness in multiplayer games relies on combining a players' *Seed* with a *Hash* from a pre-generated casino *Hash Chain*. This *Hash Chain* is created before the game room is instantiated.

To ensure transparency casino commits to the *Hash Chain* by revealing its *Last Hash* to the community of players. Once the *Last Hash* is shared, players' *Seed* is set for the players, ensuring that the Hash Chain could not have been manipulated in a way that benefits the casino.

Generating hash buffer

Given the multiplayer draw's *Hash* and *Seed*, the *Hash Buffer* is generated with the following formula:

`HMAC_SHA256(HASH,$SEED:$HASH_INDEX)`

where *Hash Index* is incremented for as many extractions as the Draw requires (see: General Concepts / Hash Buffer).

Hash

NOTE: it can be confusing that term *Hash* has a different meaning in context of *Hash Chain* and *Hash Buffer*. *Hashes* from the Chain are used to create Hmac generating the *Hash Buffer* for the given draw.

Casino pre-generates a long sequence (usually 1-10 million long) of SHA256 hashes. Starting from random 32-bytes hex string and then using the formula:

`HASH = SHA256(PREVIOUS_HASH)`

Last hash from this *Hash Chain* is not used for random numbers generation (it's intended for publishing to the players), but then each multiplayer Draw will consume the next *Hash* in a backwards fashion. I.e. the first Draw of the multiplayer Room will use the *Hash* that was used to produce the very *Last Hash*, the second draw will use the one before and so on.

Due to the nature of hashing algorithms (SHA256 in this case) - even if the current hash is revealed to the players, noone is able to decode which one was used to generate it and predict the next round RNG output.

On the other hand, once the *Hash* is revealed after the Draw is finished, everyone is able to apply SHA256 calculus and verify that this *Hash* was indeed the one to produce the one used for previous Draw, thus ensuring *Hash Chain* consistency and casino's initial *Hash Chain* commitment.

Seed

The players' *Seed* is a string with up to 64 characters that can be set in the Back-Office for a given multiplayer Room. It can only be set once the *Hash Chain* is generated.

It's up to the casino to inform players about *Last Hash* before setting players' *Seed*. Preferably it should be set to some future value declared at the moment of *Chain Hash* commitment - such as future Bitcoin block hash. Only then *Seed* will ensure that players have an influence on the randomness of the outcomes and that *Hash Chain* was not generated to casino's advantage.

Implementation

node.js

```
function random(hash, seed, cursor) {
    // Generate hmac
    const hashIndex = Math.floor(cursor / 8);
    const hmac = crypto.createHmac("sha256", hash).update(`${seed}:${hashIndex}`);

    // Read integer from the buffer
    const offset = cursor % 8;
    const result = hmac.digest().readUInt32BE(offset * 4);

    return result;
}
```

JavaScript in the Browser

Requires installing `crypto-js` (https://github.com/brix/crypto-js)

```
import CryptoJS from "crypto-js";

function random(hash, seed, cursor) {
    // Generate hmac
    const hashIndex = Math.floor(cursor / 8);
    const hmac = CryptoJS.HmacSHA256(`${seed}:${hashIndex}`, hash);

    // Read the integer and conver from signed to unsigned
    const offset = cursor % 8;
    const result = hmac.words[offset] >>> 0;
```

```
    return result;
}
```

General concepts

Algorithms described in this section apply to both single-player and multiplayer random numbers generation.

Hash Buffer

Both Single-player Round and Multiplayer Draw can require multiple random numbers to generate the gameplay. To make it possible while providing ways to prove fairness, each Round and Draw is provided a *Hash Buffer* (which is theoretically infinite). Both these game modes use different techniques for generating *Hash Buffer*, but the technique of consuming numbers from this buffer is common.

Integers extraction

Games use 4 bytes from the *Hash Buffer* to generate each requested random number. Since SHA256 provides 32 bytes, we can return 8 32-bit unsigned integers from each hash. Bytes are interpreted as unsigned integers in a Big Endian order.

Cursor and hash index

The cursor is incremented every time new random number is requested, as well as when the random number requires re-rolling (see: Generating number under a limit). When the game requests more than 8 numbers in a single round/draw (32 bytes / 4 bytes) we will increment hashIndex and generate new hash.

Generating number under a limit

It is quite common that a game needs to get a random number under a specified limit (e.q. under 100 so `[0–100)`). Multiplying integer by a float can generate a random number within a range, but it may introduce bias when converting the floating-point result to an integer.

The `unbiasedRandomInteger` function ensures a truly uniform distribution by carefully managing the range and using the modulo operation.

```
function unbiasedRandomInteger(limit) {
    // Determine the Smallest Power of 2 Greater Than or Equal to the Limit:
    let power = 1;
    while (power < limit) {
        power *= 2;
    }

    // Generate a Random Integer avoiding the bias
    let number;
    do {
        const int = random();
        number = int % power;
    } while (number >= limit);

    return number;
}
```

API

Path: `/fairness/updateClientSeed` **Method:** `POST` **Authorization:** `YES`

Attempts to update *Player Seed*. Throws error if any rounds are open on the current *Player Seed*.

Body

Name	Type		Example	Description
`clientSeed`	`string`	`REQUIRED`	`my-lucky-player-seed`	new *Player Seed*

Response

Name	Type	Example	Description
`clientSeed`	`string`	`my-lucky-player-seed`	new *Player Seed*
`serverSeedHash`	`string`	`c200626f`	hashed *Server Seed*
`nextServerSeedHash`	`string`	`31b3bb7b`	hashed next *Server Seed* (the underlying seed will be used as active *Server Seed* next time when player updates Seeds)
`nonce`	`number`	0	current nonce
`unfinishedGames`	`string[]`	[]	array of games with rounds open on active seeds (empty)

Path: `/fairness/activeRngSeeds` **Method:** `GET` **Authorization:** `YES`

Gets active *Player Seeds* for the authenticated player (no arguments required)

Response

Name	Type	Example	Description
`clientSeed`	`string`	`my-lucky-player-seed`	new *Player Seed*
`serverSeedHash`	`string`	`c200626f`	hashed *Server Seed*
`nextServerSeedHash`	`string`	`31b3bb7b`	hashed next *Server Seed* (the underlying seed will be used as active *Server Seed* next time when player updates Seeds)
`nonce`	`number`	1234	current nonce
`unfinishedGames`	`string[]`	["dice", "plinko"]	array of games with open rounds on the active seeds (these rounds need to be completed in order to update *Player Seed*

Path: `/fairness/roundRngState` **Method:** `GET` **Authorization:** `NO`

Returns Provably Fair round RNG state for a given `roundId`.

Query Parameters

Name	Type		Example	Description
`roundId`	`string`	`REQUIRED`	`d615412f-1e9b-4f0c-9b6c-494d68b234a8`	roundId

Response

Name	Type	Example	Description
`clientSeed`	`string`	`my-lucky-player-seed`	*Player Seed* that this round was played on
`serverSeedHash`	`string`	`31b3bb7b`	hashed *Server Seed* that this round was played on
`nextServerSeedHash`	`string`	`73a29388`	hashed next *Server Seed* (the underlying seed will be used as active *Server Seed* next time when player updates Seeds)
`nonce`	`number`	231	nonce for the given round
`serverSeed`	`string`	`f8bc155a`	present only if server seed was revealed
`status`	`string`	"active" or "revealed"	seeds status

Path: `/fairness/unhashServerSeed` **Method:** `GET` **Authorization:** `NO`

Returns unhashed server seed if this seed was revealed (i.e. player rotated the Seeds).

Query Parameters

Name	Type		Example	Description
`serverSeedHash`	`string`	`REQUIRED`	`31b3bb7b`	hash to reveal

Response

Name	Type	Example	Description
`serverSeed`	`string`	`f8bc155a`	unhashed server seed - present only if server seed was revealed